

```

Int i,it,j,ncof=mm+kk+1,npt=NPFAC*ncof;
Number of points where function is evaluated, i.e., fineness of the mesh.
Doub devmax,e,hth,power,sum;
VecDoub bb(npt),coff(ncof),ee(npt),fs(npt),wt(npt),xs(npt);
MatDoub u(npt,ncof);
Ratfn ratbest(coff,mm+1,kk+1);
dev=BIG;
for (i=0;i<npt;i++) {                               Fill arrays with mesh abscissas and function val-
    if (i < (npt/2)-1) {                               ues.
        hth=PI02*i/(npt-1.0);                         At each end, use formula that minimizes round-
        xs[i]=a+(b-a)*SQR(sin(hth));                   off sensitivity.
    } else {
        hth=PI02*(npt-i)/(npt-1.0);
        xs[i]=b-(b-a)*SQR(sin(hth));
    }
    fs[i]=fn(xs[i]);
    wt[i]=1.0;                                         In later iterations we will adjust these weights to
    ee[i]=1.0;                                         combat the largest deviations.
}
e=0.0;
for (it=0;it<MAXIT;it++) {                           Loop over iterations.
    for (i=0;i<npt;i++) {                             Set up the "design matrix" for the least-squares
        power=wt[i];                                  fit.
        bb[i]=power*(fs[i]+SIGN(e,ee[i]));
        Key idea here: Fit to  $fn(x) + e$  where the deviation is positive, to  $fn(x) - e$  where
        it is negative. Then  $e$  is supposed to become an approximation to the equal-ripple
        deviation.
        for (j=0;j<mm+1;j++) {
            u[i][j]=power;
            power *= xs[i];
        }
        power = -bb[i];
        for (j=mm+1;j<ncof;j++) {
            power *= xs[i];
            u[i][j]=power;
        }
    }
    SVD svd(u);                                       Singular value decomposition.
    svd.solve(bb,coff);
    In especially singular or difficult cases, one might here edit the singular values, replacing
    small values by zero in w[0..ncof-1].
    devmax=sum=0.0;
    Ratfn rat(coff,mm+1,kk+1);
    for (j=0;j<npt;j++) {                             Tabulate the deviations and revise the weights.
        ee[j]=rat(xs[j])-fs[j];
        wt[j]=abs(ee[j]);                             Use weighting to emphasize most deviant points.
        sum += wt[j];
        if (wt[j] > devmax) devmax=wt[j];
    }
    e=sum/npt;                                       Update  $e$  to be the mean absolute deviation.
    if (devmax <= dev) {                             Save only the best coefficient set found.
        ratbest = rat;
        dev=devmax;
    }
    cout << " ratlsq iteration= " << it;
    cout << "   max error= " << setw(10) << devmax << endl;
}
return ratbest;
}

```

Figure 5.13.1 shows the discrepancies for the first five iterations of `ratlsq` when it is applied to find the $m = k = 4$ rational fit to the function $f(x) = \cos x / (1 + e^x)$ in the interval $(0, \pi)$. One sees that after the first iteration, the results are virtually as good as the

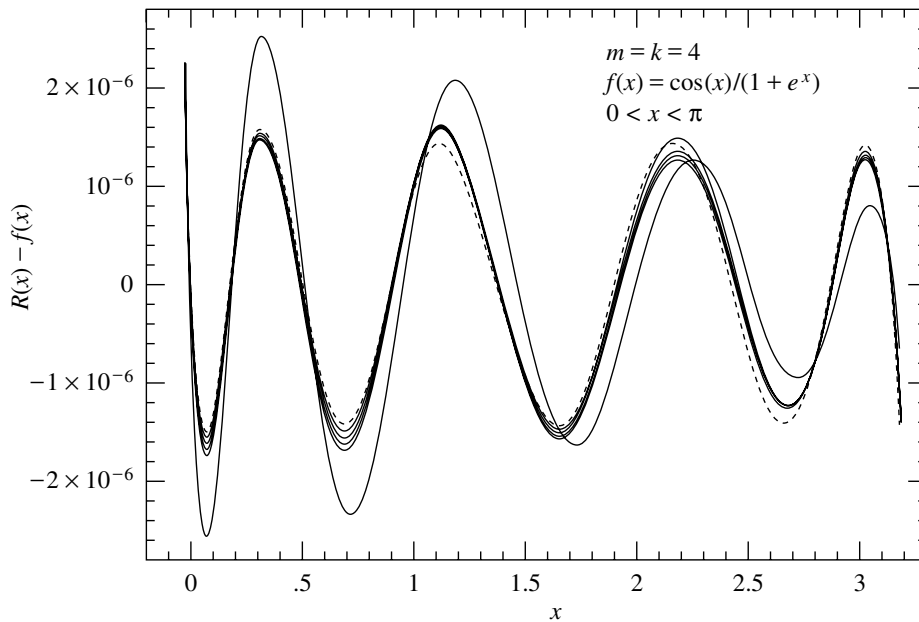


Figure 5.13.1. Solid curves show deviations $r(x)$ for five successive iterations of the routine `ratlsq` for an arbitrary test problem. The algorithm does not converge to exactly the minimax solution (shown as the dotted curve). But, after one iteration, the discrepancy is a small fraction of the last significant bit of accuracy.

minimax solution. The iterations do not converge in the order that the figure suggests. In fact, it is the second iteration that is best (has smallest maximum deviation). The routine `ratlsq` accordingly returns the best of its iterations, not necessarily the last one; there is no advantage in doing more than five iterations.

CITED REFERENCES AND FURTHER READING:

Ralston, A. and Wilf, H.S. 1960, *Mathematical Methods for Digital Computers* (New York: Wiley), Chapter 13.[1]

5.14 Evaluation of Functions by Path Integration

In computer programming, the technique of choice is not necessarily the most efficient, or elegant, or fastest executing one. Instead, it may be the one that is quick to implement, general, and easy to check.

One sometimes needs only a few, or a few thousand, evaluations of a special function, perhaps a complex-valued function of a complex variable, that has many different parameters, or asymptotic regimes, or both. Use of the usual tricks (series, continued fractions, rational function approximations, recurrence relations, and so forth) may result in a patchwork program with tests and branches to different formulas. While such a program may be highly efficient in execution, it is often not the shortest way to the answer from a standing start.

A different technique of considerable generality is direct integration of a function's defining differential equation — an *ab initio* integration for each desired function value — along a path in the complex plane if necessary. While this may at first seem like swatting a fly with a golden brick, it turns out that when you already have the brick, and the fly is asleep right under it, all you have to do is let it fall!

As a specific example, let us consider the complex hypergeometric function ${}_2F_1(a, b, c; z)$, which is defined as the analytic continuation of the so-called hypergeometric series,

$$\begin{aligned} {}_2F_1(a, b, c; z) = & 1 + \frac{ab}{c} \frac{z}{1!} + \frac{a(a+1)b(b+1)}{c(c+1)} \frac{z^2}{2!} + \cdots \\ & + \frac{a(a+1) \cdots (a+j-1)b(b+1) \cdots (b+j-1)}{c(c+1) \cdots (c+j-1)} \frac{z^j}{j!} + \cdots \end{aligned} \quad (5.14.1)$$

The series converges only within the unit circle $|z| < 1$ (see [1]), but one's interest in the function is often not confined to this region.

The hypergeometric function ${}_2F_1$ is a solution (in fact *the* solution that is regular at the origin) of the hypergeometric differential equation, which we can write as

$$z(1-z)F'' = abF - [c - (a+b+1)z]F' \quad (5.14.2)$$

Here prime denotes d/dz . One can see that the equation has regular singular points at $z = 0, 1$, and ∞ . Since the desired solution is regular at $z = 0$, the values 1 and ∞ will in general be branch points. If we want ${}_2F_1$ to be a single-valued function, we must have a branch cut connecting these two points. A conventional position for this cut is along the positive real axis from 1 to ∞ , though we may wish to keep open the possibility of altering this choice for some applications.

Our golden brick consists of a collection of routines for the integration of sets of ordinary differential equations, which we will develop in detail later, in Chapter 17. For now, we need only a high-level, “black-box” routine that integrates such a set from initial conditions at one value of a (real) independent variable to final conditions at some other value of the independent variable, while automatically adjusting its internal stepsize to maintain some specified accuracy. That routine is called `Odeint` and, in one particular invocation, it calculates its individual steps with a sophisticated Bulirsch-Stoer technique.

Suppose that we know values for F and its derivative F' at some value z_0 , and that we want to find F at some other point z_1 in the complex plane. The straight-line path connecting these two points is parametrized by

$$z(s) = z_0 + s(z_1 - z_0) \quad (5.14.3)$$

with s a real parameter. The differential equation (5.14.2) can now be written as a set of two first-order equations,

$$\begin{aligned} \frac{dF}{ds} &= (z_1 - z_0)F' \\ \frac{dF'}{ds} &= (z_1 - z_0) \left(\frac{abF - [c - (a+b+1)z]F'}{z(1-z)} \right) \end{aligned} \quad (5.14.4)$$

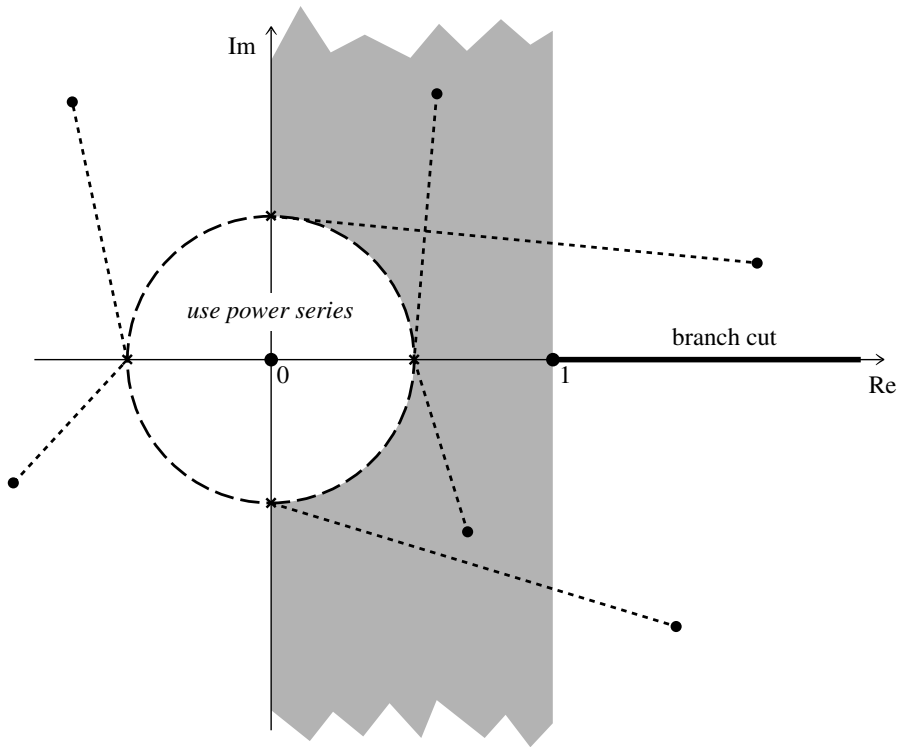


Figure 5.14.1. Complex plane showing the singular points of the hypergeometric function, its branch cut, and some integration paths from the circle $|z| = 1/2$ (where the power series converges rapidly) to other points in the plane.

to be integrated from $s = 0$ to $s = 1$. Here F and F' are to be viewed as two independent complex variables. The fact that prime means d/dz can be ignored; it will emerge as a consequence of the first equation in (5.14.4). Moreover, the real and imaginary parts of equation (5.14.4) define a set of four *real* differential equations, with independent variable s . The complex arithmetic on the right-hand side can be viewed as mere shorthand for how the four components are to be coupled. It is precisely this point of view that gets passed to the routine `Odeint`, since it knows nothing of either complex functions or complex independent variables.

It remains only to decide where to start, and what path to take in the complex plane, to get to an arbitrary point z . This is where consideration of the function's singularities, and the adopted branch cut, enter. Figure 5.14.1 shows the strategy that we adopt. For $|z| \leq 1/2$, the series in equation (5.14.1) will in general converge rapidly, and it makes sense to use it directly. Otherwise, we integrate along a straight-line path from one of the starting points $(\pm 1/2, 0)$ or $(0, \pm 1/2)$. The former choices are natural for $0 < \text{Re}(z) < 1$ and $\text{Re}(z) < 0$, respectively. The latter choices are used for $\text{Re}(z) > 1$, above and below the branch cut; the purpose of starting away from the real axis in these cases is to avoid passing too close to the singularity at $z = 1$ (see Figure 5.14.1). The location of the branch cut is *defined* by the fact that our adopted strategy never integrates across the real axis for $\text{Re}(z) > 1$.

An implementation of this algorithm is given in §6.13 as the routine *hypgeo*.

A number of variants on the procedure described thus far are possible and easy to program. If successively called values of z are close together (with identical values of a , b , and c), then you can save the state vector (F, F') and the corresponding value of z on each call, and use these as starting values for the next call. The incremental integration may then take only one or two steps. Avoid integrating across the branch cut unintentionally: The function value will be “correct,” but not the one you want.

Alternatively, you may wish to integrate to some position z by a dog-leg path that *does* cross the real axis $\operatorname{Re}(z) > 1$, as a means of *moving* the branch cut. For example, in some cases you might want to integrate from $(0, 1/2)$ to $(3/2, 1/2)$, and go from there to any point with $\operatorname{Re}(z) > 1$ — with either sign of $\operatorname{Im}z$. (If you are, for example, finding roots of a function by an iterative method, you do not want the integration for nearby values to take different paths around a branch point. If it does, your root-finder will see discontinuous function values and will likely not converge correctly!)

In any case, be aware that a loss of numerical accuracy can result if you integrate through a region of large function value on your way to a final answer where the function value is small. (For the hypergeometric function, a particular case of this is when a and b are both large and positive, with c and $x \gtrsim 1$.) In such cases, you’ll need to find a better dog-leg path.

The general technique of evaluating a function by integrating its differential equation in the complex plane can also be applied to other special functions. For example, the complex Bessel function, Airy function, Coulomb wave function, and Weber function are all special cases of the *confluent hypergeometric function*, with a differential equation similar to the one used above (see, e.g., [1] §13.6, for a table of special cases). The confluent hypergeometric function has no singularities at finite z : That makes it easy to integrate. However, its essential singularity at infinity means that it can have, along some paths and for some parameters, highly oscillatory or exponentially decreasing behavior: That makes it hard to integrate. Some case-by-case judgment (or experimentation) is therefore required.

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>. [1]

6.0 Introduction

There is nothing particularly special about a *special function*, except that some person in authority or a textbook writer (not the same thing!) has decided to bestow the moniker. Special functions are sometimes called *higher transcendental functions* (higher than what?) or *functions of mathematical physics* (but they occur in other fields also) or *functions that satisfy certain frequently occurring second-order differential equations* (but not all special functions do). One might simply call them “useful functions” and let it go at that. The choice of which functions to include in this chapter is highly arbitrary.

Commercially available program libraries contain many special function routines that are intended for users who will have no idea what goes on inside them. Such state-of-the-art black boxes are often very messy things, full of branches to completely different methods depending on the value of the calling arguments. Black boxes have, or should have, careful control of accuracy, to some stated uniform precision in all regimes.

We will not be quite so fastidious in our examples, in part because we want to illustrate techniques from Chapter 5, and in part because we *want* you to understand what goes on in the routines presented. Some of our routines have an accuracy parameter that can be made as small as desired, while others (especially those involving polynomial fits) give only a certain stated accuracy, one that we believe is serviceable (usually, but not always, close to double precision). We do *not* certify that the routines are perfect black boxes. We do hope that, if you ever encounter trouble in a routine, you will be able to diagnose and correct the problem on the basis of the information that we have given.

In short, the special function routines of this chapter are meant to be used — we use them all the time — but we also want you to learn from their inner workings.

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>.

- Andrews, G.E., Askey, R., and Roy, R. 1999, *Special Functions* (Cambridge, UK: Cambridge University Press).
- Thompson, W.J. 1997, *Atlas for Computing Mathematical Functions* (New York: Wiley-Interscience).
- Spanier, J., and Oldham, K.B. 1987, *An Atlas of Functions* (Washington: Hemisphere Pub. Corp.).
- Wolfram, S. 2003, *The Mathematica Book*, 5th ed. (Champaign, IL: Wolfram Media).
- Hart, J.F., et al. 1968, *Computer Approximations* (New York: Wiley).
- Hastings, C. 1955, *Approximations for Digital Computers* (Princeton: Princeton University Press).
- Luke, Y.L. 1975, *Mathematical Functions and Their Approximations* (New York: Academic Press).

6.1 Gamma Function, Beta Function, Factorials, Binomial Coefficients

The gamma function is defined by the integral

$$\Gamma(z) = \int_0^{\infty} t^{z-1} e^{-t} dt \quad (6.1.1)$$

When the argument z is an integer, the gamma function is just the familiar factorial function, but offset by 1:

$$n! = \Gamma(n + 1) \quad (6.1.2)$$

The gamma function satisfies the recurrence relation

$$\Gamma(z + 1) = z \Gamma(z) \quad (6.1.3)$$

If the function is known for arguments $z > 1$ or, more generally, in the half complex plane $\text{Re}(z) > 1$, it can be obtained for $z < 1$ or $\text{Re}(z) < 1$ by the reflection formula

$$\Gamma(1 - z) = \frac{\pi}{\Gamma(z) \sin(\pi z)} = \frac{\pi z}{\Gamma(1 + z) \sin(\pi z)} \quad (6.1.4)$$

Notice that $\Gamma(z)$ has a pole at $z = 0$ and at all negative integer values of z .

There are a variety of methods in use for calculating the function $\Gamma(z)$ numerically, but none is quite as neat as the approximation derived by Lanczos [1]. This scheme is entirely specific to the gamma function, seemingly plucked from thin air. We will not attempt to derive the approximation, but only state the resulting formula: For certain choices of rational γ and integer N , and for certain coefficients $c_1, c_2, \dots, c_N$, the gamma function is given by

$$\begin{aligned} \Gamma(z + 1) &= (z + \gamma + \frac{1}{2})^{z + \frac{1}{2}} e^{-(z + \gamma + \frac{1}{2})} \\ &\times \sqrt{2\pi} \left[c_0 + \frac{c_1}{z + 1} + \frac{c_2}{z + 2} + \dots + \frac{c_N}{z + N} + \epsilon \right] \quad (z > 0) \end{aligned} \quad (6.1.5)$$

You can see that this is a sort of take-off on Stirling's approximation, but with a series of corrections that take into account the first few poles in the left complex plane. The constant c_0 is very nearly equal to 1. The error term is parametrized by

ϵ . For $N = 14$, and a certain set of c 's and γ (calculated by P. Godfrey), the error is smaller than $|\epsilon| < 10^{-15}$. Even more impressive is the fact that, with these same constants, the formula (6.1.5) applies for the *complex* gamma function, *everywhere in the half complex plane* $\operatorname{Re} z > 0$, achieving almost the same accuracy as on the real line.

It is better to implement $\ln \Gamma(x)$ than $\Gamma(x)$, since the latter will overflow at quite modest values of x . Often the gamma function is used in calculations where the large values of $\Gamma(x)$ are divided by other large numbers, with the result being a perfectly ordinary value. Such operations would normally be coded as subtraction of logarithms. With (6.1.5) in hand, we can compute the logarithm of the gamma function with two calls to a logarithm and a few dozen arithmetic operations. This makes it not much more difficult than other built-in functions that we take for granted, such as $\sin x$ or e^x :

```
Doub gammln(const Doub xx) { gamma.h
Returns the value  $\ln[\Gamma(xx)]$  for  $xx > 0$ .
    Int j;
    Doub x,tmp,y,ser;
    static const Doub cof[14]={57.1562356658629235,-59.5979603554754912,
14.1360979747417471,-0.491913816097620199,.339946499848118887e-4,
.465236289270485756e-4,-.983744753048795646e-4,.158088703224912494e-3,
-.210264441724104883e-3,.217439618115212643e-3,-.164318106536763890e-3,
.844182239838527433e-4,-.261908384015814087e-4,.368991826595316234e-5};
    if (xx <= 0) throw("bad arg in gammln");
    y=x=xx;
    tmp = x+5.242187500000000000; Rational 671/128.
    tmp = (x+0.5)*log(tmp)-tmp;
    ser = 0.999999999999997092;
    for (j=0;j<14;j++) ser += cof[j]/++y;
    return tmp+log(2.5066282746310005*ser/x);
}
```

How shall we write a routine for the factorial function $n!$? Generally the factorial function will be called for small integer values, and in most applications the same integer value will be called for many times. It is obviously inefficient to call `exp(gammln(n+1.))` for each required factorial. Better is to initialize a static table on the first call, and do a fast lookup on subsequent calls. The fixed size 171 for the table is because $170!$ is representable as an IEEE double precision value, but $171!$ overflows. It is also sometimes useful to know that factorials up to $22!$ have exact double precision representations (52 bits of mantissa, not counting powers of two that are absorbed into the exponent), while $23!$ and above are represented only approximately.

```
Doub factrl(const Int n) { gamma.h
Returns the value  $n!$  as a floating-point number.
    static VecDoub a(171);
    static Bool init=true;
    if (init) {
        init = false;
        a[0] = 1.;
        for (Int i=1;i<171;i++) a[i] = i*a[i-1];
    }
    if (n < 0 || n > 170) throw("factrl out of range");
    return a[n];
}
```

More useful in practice is a function returning the log of a factorial, which doesn't have overflow issues. The size of the table of logarithms is whatever you can afford in space and initialization time. The value $\text{NTOP} = 2000$ should be increased if your integer arguments are often larger.

```
gamma.h  Doub factln(const Int n) {
Returns ln(n!).
    static const Int NTOP=2000;
    static VecDoub a(NTOP);
    static Bool init=true;
    if (init) {
        init = false;
        for (Int i=0;i<NTOP;i++) a[i] = gammln(i+1.);
    }
    if (n < 0) throw("negative arg in factln");
    if (n < NTOP) return a[n];
    return gammln(n+1.);           Out of range of table.
}
```

The binomial coefficient is defined by

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad 0 \leq k \leq n \quad (6.1.6)$$

A routine that takes advantage of the tables stored in `factrl` and `factln` is

```
gamma.h  Doub bico(const Int n, const Int k) {
Returns the binomial coefficient  $\binom{n}{k}$  as a floating-point number.
    if (n<0 || k<0 || k>n) throw("bad args in bico");
    if (n<171) return floor(0.5+factrl(n)/(factrl(k)*factrl(n-k)));
    return floor(0.5+exp(factln(n)-factln(k)-factln(n-k)));
    The floor function cleans up roundoff error for smaller values of n and k.
}
```

If your problem requires a series of related binomial coefficients, a good idea is to use recurrence relations, for example,

$$\begin{aligned} \binom{n+1}{k} &= \frac{n+1}{n-k+1} \binom{n}{k} = \binom{n}{k} + \binom{n}{k-1} \\ \binom{n}{k+1} &= \frac{n-k}{k+1} \binom{n}{k} \end{aligned} \quad (6.1.7)$$

Finally, turning away from the combinatorial functions with integer-valued arguments, we come to the beta function,

$$B(z, w) = B(w, z) = \int_0^1 t^{z-1} (1-t)^{w-1} dt \quad (6.1.8)$$

which is related to the gamma function by

$$B(z, w) = \frac{\Gamma(z)\Gamma(w)}{\Gamma(z+w)} \quad (6.1.9)$$

hence